



Desarrollo Web

FACULTAD CIENCIAS DE LA INGENIERÍA, CARRERA SOFTWARE -
PRESENCIAL

Unidad 1

LENGUAJE DEL LADO DEL CLIENTE - FRONT END

Tema 3

CSS

MSc. Josué Guevara
rguevarar@unemi.edu.ec

Subtemas

Subtema 1: CSS y HTML.

Subtema 2: Semántica (Etiquetas y elementos).

Subtema 3: Formularios, transiciones y animaciones.

Subtema 4: Practica.

Objetivo

Conocer la estructura de trabajo cuando se combina el desarrollo entre HTML y CSS, además de las etiquetas base y elementos principales de diseño.

Introducción

Tema:

¿Qué hace que un formulario sea fácil de usar?

¿Cuál ha sido el efecto visual más interesante que han visto en una página web?

Introducción

CSS(Cascading Style Sheets), es esencial para dar estilo y formato a las páginas web, separando el contenido del diseño visual. Junto con el HTML, CSS permite definir la apariencia de los elementos en un sitio, desde colores y tipografías hasta la disposición de los componentes en la página.

CSS y HTML – La Pareja Perfecta

HTML y CSS trabajan juntos para construir cada página web que visitas. Sin CSS, la web sería un documento de texto sin formato ni personalidad.

HTML 🦴

La **estructura y el esqueleto** de tu página. Define qué hay: párrafos, encabezados, imágenes, listas y formularios. Es el contenido en bruto.

CSS 🎨

El **estilo, diseño y presentación**. Controla colores, tipografías, tamaños, posiciones y animaciones. Convierte el esqueleto en algo visualmente atractivo.

Juntos 🚀

La combinación perfecta: HTML aporta el **qué** y CSS aporta el **cómo se ve**. Sin CSS tu web sería solo texto plano sin estructura visual.



Selectores: ¿A Quién le Hablamos?

Los selectores son el mecanismo con el que CSS identifica qué elementos del HTML deben recibir un estilo determinado. Elegir el selector correcto es fundamental para un código limpio y mantenible.

1

Selector de Etiqueta

Aplica estilos a **todos** los elementos de ese tipo en la página.

```
p { color: blue; }
```

2

Selector de Clase

Aplica estilos a **cualquier elemento** que tenga ese atributo `class`. Son **reutilizables**.

```
.tarjeta { padding: 1rem; }
```

3

Selector de ID

Apunta a un **único elemento** en toda la página identificado con `id`. Son irrepetibles.

```
#cabecera { height: 100vh; }
```



Diferencia clave: Las **clases** (`.nombre`) son reutilizables y pueden aplicarse a múltiples elementos. Los **IDs** (`#nombre`) son únicos: solo debe existir uno por página. Usa clases por defecto y reserva los IDs para casos muy específicos.

El Modelo de Caja: Cada Elemento es una Caja

En CSS, **todos los elementos HTML son cajas rectangulares**. El modelo de caja describe las capas que rodean cada elemento y determina cómo se calcula su tamaño y espacio en la página.

→ Contenido

El núcleo: texto, imágenes o cualquier otro elemento hijo.
Sus dimensiones se controlan con `width` y `height`.

→ Padding (Relleno)

Espacio **interior** entre el contenido y el borde. Aumenta el área visible del elemento sin afectar el espacio entre elementos.

→ Border (Borde)

La línea que **envuelve** el padding y el contenido. Puede tener grosor, estilo y color personalizados.

→ Margin (Margen)

Espacio **exterior** que separa esta caja de los elementos adyacentes. Es transparente y no tiene color propio.

¡Truco Esencial!

Por defecto, `width` solo mide el contenido. Usa esta regla global para evitar cálculos complicados:

```
* {  
  box-sizing:  
    border-box;  
}
```

Con `border-box`, el `width` **incluye** el padding y el borde, haciendo el diseño mucho más predecible.

Diseñando con Flexbox

Flexbox (Flexible Box Layout) es un modelo de diseño **unidimensional** que revolucionó la forma de maquetar en CSS. Permite distribuir y alinear elementos en una fila o columna de manera eficiente, incluso cuando sus tamaños son desconocidos o dinámicos.



display: flex

Activa el contexto Flexbox en el elemento **contenedor**. Sus hijos directos se convierten automáticamente en *flex items*.



justify-content

Controla la alineación de los elementos en el **eje principal** (horizontal por defecto): `center`, `space-between`, `flex-end`...



align-items

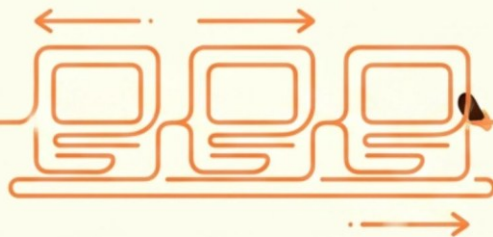
Controla la alineación en el **eje transversal** (vertical por defecto). El valor `center` resuelve el clásico problema del centrado vertical.



¡Adiós a los floats! Flexbox resuelve de forma elegante problemas históricos del CSS: centrar verticalmente, igualar alturas de columnas y distribuir espacio de forma proporcional. Es la herramienta estándar para layouts modernos.

Layout Moderno: Flexbox y Grid

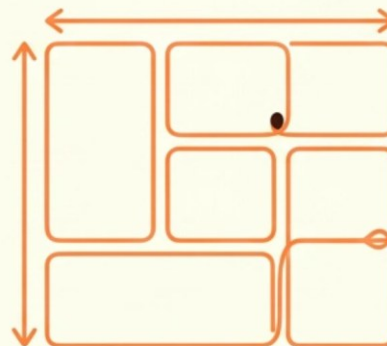
Flexbox: Unidimensional



Casos de uso: Barra de navegación, Fila de tarjetas, Grupo de botones

Propiedades: flex-direction, justify-content, align-items, gap

Grid: Bidimensional



Casos de uso: Diseño de página, Galería de imágenes

Propiedades: grid-template-columns, minmax, auto-fit, gap

Regla práctica: **Flexbox** para alinear elementos en una fila o columna; **Grid** para estructurar regiones de página en dos dimensiones.

Pueden combinarse: Grid organiza la página, Flexbox organiza el interior de cada celda.

Decisión de Ingeniería: Flexbox vs Grid vs Container Queries

Criterio	Flexbox	Grid	Container Queries
Dimensiones	1D (fila o columna)	2D (filas y columnas)	1D o 2D según necesidad
Caso ideal	Nav, grupo de botones, tarjeta interna	Layout de página, galería, dashboard	Componentes reutilizables independientes del viewport
Soporte	Universal	Universal	Amplio (verificar en caniuse)
Curva de aprendizaje	Baja	Media	Media-alta
Trade-off	Complejo en 2D	Sobre-ingeniería para listas simples	Requiere declarar container-type

❏ No hay ganador universal: usa el mecanismo que reduce la complejidad del código para ese contexto específico.

Responsive Design

El diseño responsivo garantiza que una página web se vea y funcione correctamente en cualquier dispositivo. El enfoque **mobile-first** parte del diseño para pantallas pequeñas y va añadiendo complejidad para pantallas más grandes.

1

Mobile First

Escribe estilos base para móvil sin media queries

2

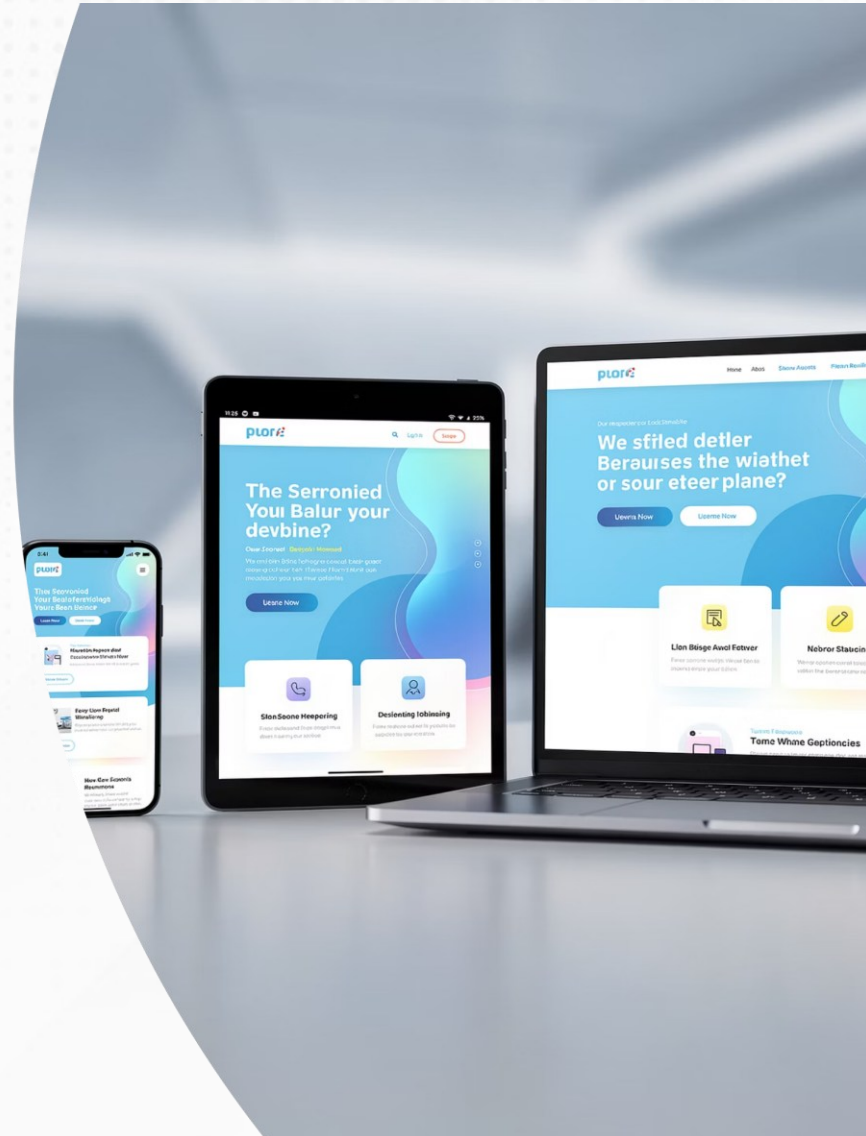
Tablet

```
@media (min-width: 768px) {  
  .grid { columns: 2; }  
}
```

3

Desktop

```
@media (min-width: 1024px) {  
  .grid { columns: 3; }  
}
```



Variables CSS (Custom Properties)

Las **variables CSS** permiten definir valores reutilizables en una sola ubicación. Esto facilita el mantenimiento, la consistencia visual y la implementación de temas dinámicos (dark/light mode).

Definición en :root

```
:root {  
  --color-primario: #2589C9;  
  --color-oscuro: #063E5F;  
  --fuente-base: 1rem;  
  --radio-borde: 8px;  
  --sombra: 0 4px 12px  
    rgba(0,0,0,0.1);  
}
```

Uso práctico

```
.boton {  
  background: var(--color-primario);  
  border-radius: var(--radio-borde);  
  font-size: var(--fuente-base);  
  box-shadow: var(--sombra);  
}  
  
.boton:hover {  
  background: var(--color-oscuro);  
}
```

- ✓ Cambiar un solo valor en `:root` actualiza toda la interfaz al instante — ideal para design tokens.

Pseudoclases y Pseudoelementos

Las **pseudoclases** aplican estilos basados en el *estado* del elemento; los **pseudoelementos** permiten estilizar *partes virtuales* del elemento o insertar contenido decorativo sin modificar el HTML.

Pseudoclases más usadas

```
a:hover {  
  color: #75bae6ff;  
  text-decoration: underline;  
}  
  
input:focus {  
  outline: 2px solid #2589C9;  
  border-color: #2589C9;  
}  
  
li:nth-child(odd) {  
  background: #e5f6ff;  
}
```

Pseudoelementos ::before y ::after

```
.seccion::before {  
  content: "";  
  display: block;  
  width: 48px;  
  height: 4px;  
  background: #2589C9;  
  margin-bottom: 12px;  
}  
  
p::first-letter {  
  font-size: 2em;  
  font-weight: bold;  
  color: #063E5F;  
}
```



Los pseudoelementos `::before` y `::after` son poderosos para añadir decoraciones sin ensuciar el HTML.

Animaciones y Transiciones

CSS permite crear movimiento e interacción sin JavaScript. Las **transiciones** animan cambios de un estado a otro; las **animaciones con @keyframes** permiten secuencias complejas y en bucle.

Transición suave

```
.boton {  
  transition: all 0.3s ease;  
}  
.boton:hover {  
  transform: translateY(-4px);  
  box-shadow: var(--sombra);  
}
```

Animación @keyframes

```
@keyframes aparecer {  
  from { opacity: 0;  
         transform: translateY(20px); }  
  to   { opacity: 1;  
         transform: translateY(0); }  
}  
.tarjeta {  
  animation: aparecer 0.5s ease;  
}
```

Transformaciones 2D

```
.icono:hover {  
  transform: rotate(45deg)  
            scale(1.2);  
}  
.img {  
  transform: skewX(-10deg);  
}
```



Accesibilidad en CSS

El diseño accesible garantiza que todos los usuarios — incluyendo personas con discapacidades visuales o motoras — puedan usar tu interfaz. CSS tiene un papel fundamental en la accesibilidad percibida.



Contraste de Color

La relación de contraste entre texto y fondo debe ser al menos **4.5:1** para texto normal (WCAG AA). Usa herramientas como *Colour Contrast Checker* para verificarlo.



Indicadores de Foco

Nunca elimines el `outline` sin reemplazarlo. Los usuarios de teclado dependen de él para navegar. Personalízalo con `:focus-visible` para mantener estética y accesibilidad.



Legibilidad

Usa tamaños mínimos de **16px** para texto de cuerpo, interlineado de al menos **1.5** y evita bloques de texto demasiado anchos (máximo 70-80 caracteres por línea).



Preferir `rem` sobre `px` respeta el tamaño base configurado por el usuario en su navegador.

Síndrome de la hoja en blanco

Es aprender a buscar información y/o inspiración para combatir el síndrome de la hoja en blanco. Ejemplo:

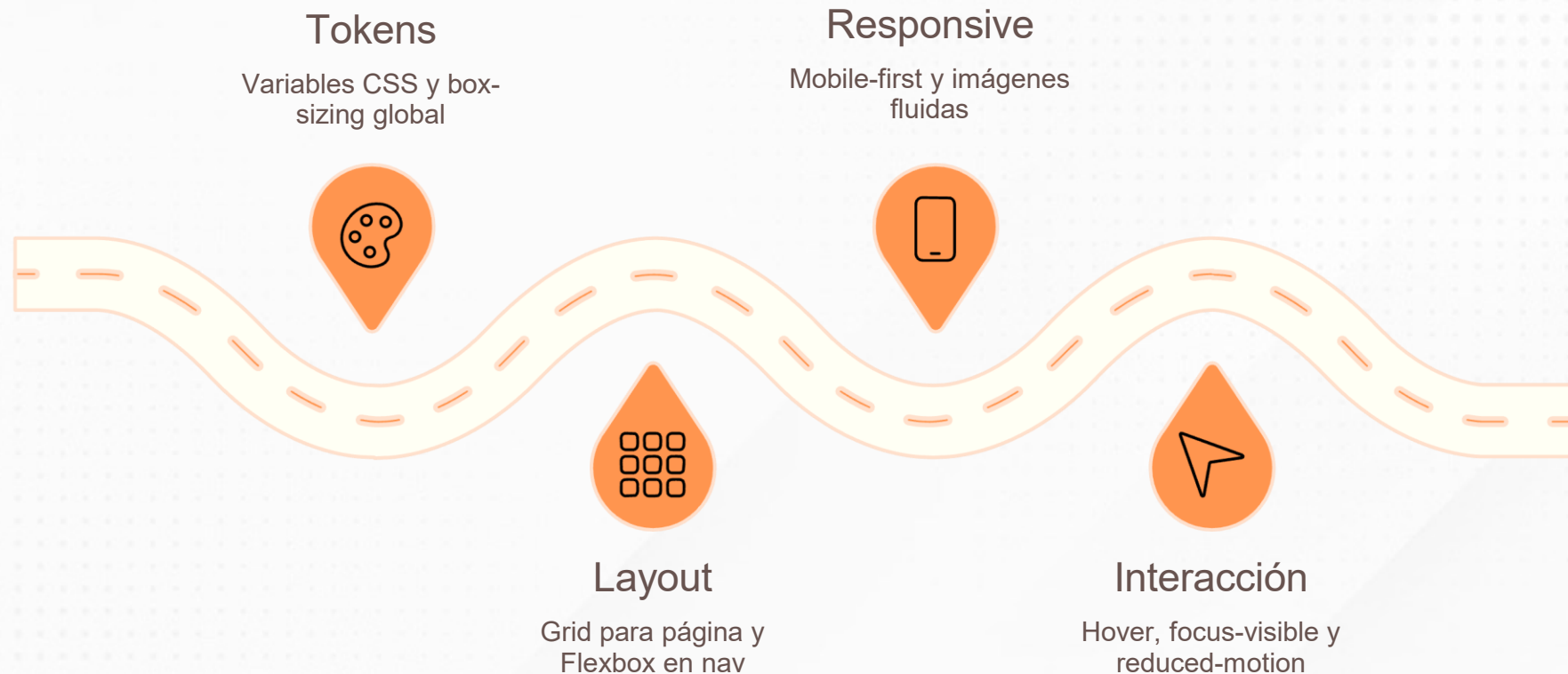
Galería:

Awwwards.- Organiza, clasifica y premia los sitios:

No es copiar e imitar otras páginas, es buscar inspiración en otras fuentes para poder solucionar nuestro problema.

Caso Integrado: Página de Evento

Diseñar la página de evento con tokens, Grid/Flexbox, navegación adaptable, formulario accesible y microinteracciones.



📌 Cada etapa es incremental: no avanzas a la siguiente hasta que la anterior funciona en los tres breakpoints.

Cierre Técnico: 5 Ideas claves

01 · Tokens primero

Define variables antes de escribir cualquier componente. Son tu contrato de diseño.

02 · Mobile-first siempre

Los estilos base son para el contexto más restringido; las media queries amplían, no corrigen.

03 · Grid + Flexbox se complementan

No compiten. Grid estructura la página; Flexbox organiza el interior de cada región.

04 · Accesibilidad no es opcional

`:focus-visible`, `contraste` y `prefers-reduced-motion` son requisitos de ingeniería, no de diseño.

05 · Anima solo compositor

`opacity` y `transform` son baratos. Animar `width`, `height` o `top` es costoso para el navegador.

Recursos útiles

- <https://developer.mozilla.org/es/docs/Web/HTML>
- <https://developer.mozilla.org/es/docs/Web/CSS>
- <https://www.w3.org/WAI/standards-guidelines/wcag/>

BIBLIOGRAFÍA

- Mastermarketing-valencia.com. (2019). ¿Cuándo nació Internet? Historia y evolución. Recuperado de <https://n9.cl/hlnxf>
- Mastermarketing-valencia.com. (2019). Infografía-La web Historia. Recuperado de <https://n9.cl/u7bct>
- Significado de Web. (2021). En Significados. Recuperado de <https://n9.cl/5gpv4>
- Schiaffarino, A. (2019). Modelo cliente servidor. Infranetworking. Recuperado de <https://n9.cl/bee5>
- ARQUITECTURA CLIENTE-SERVIDOR. (2021). Nota Tecnológica. Recuperado de <https://n9.cl/kc28a>
- Berners-Lee, T. (2000). Weaving the web: The original design and ultimate destiny of the World Wide Web. Harper Business.
- Tanenbaum, A. S., & van Steen, M. (2001). Distributed systems: Principles and paradigms (1st ed.). Pearson.
- Robbins, J. N. (2012). Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics (4th ed.). O'Reilly Media.